

UF Herbert Wertheim College of Engineering
UNIVERSITY of FLORIDA

EEL4712 Digital Design

Lecture 27 – Introduction to Assertions and Basic System Verilog Assertions (SVA)

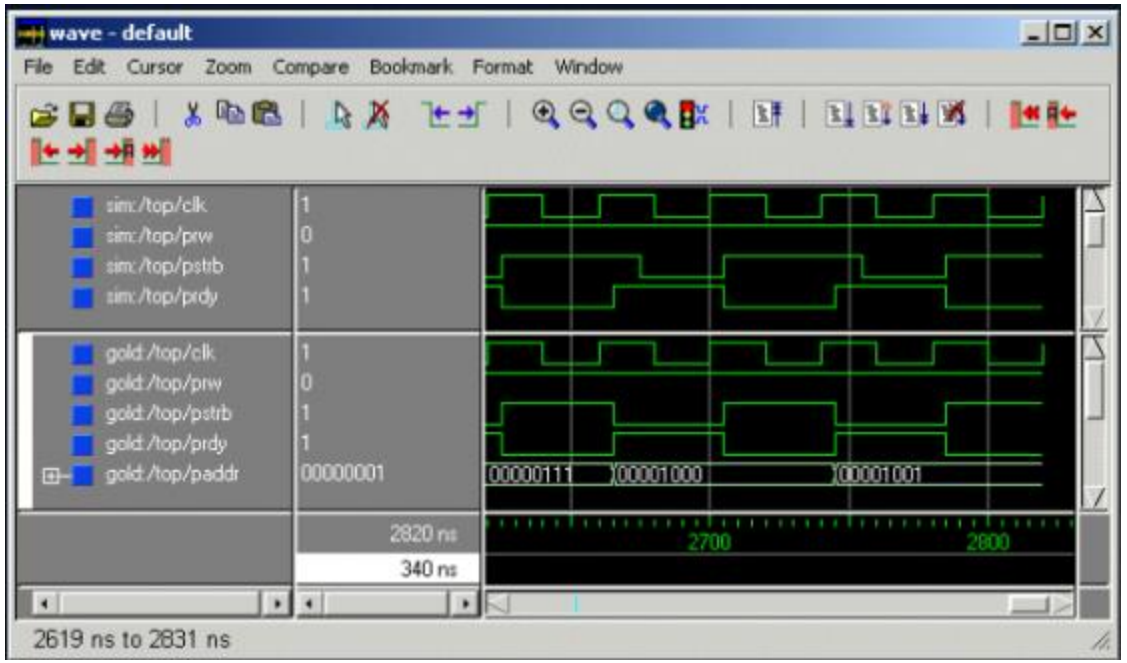
Rickard Ewetz
Spring 2026

Agenda

1. Why Assertions?
2. Immediate vs. Concurrent Assertions
3. Syntax of Concurrent Assertions
4. Implication operator
5. Code Examples on Slides
6. Live Tutorial

Why Assertions?

- Simulation alone shows waveforms, but does not automatically check correctness
- Assertions are executable specifications
- Assertions monitor behavior during simulation
- They help catch bugs early and document expected behavior
- Assertions improve debug efficiency and verification quality



Instead of manually checking outputs, assertions do the watching for you!

What is an Assertion?

- An assertion is a statement that a required property must hold
- If the property fails, the simulator reports an error
- Assertions can be placed in:
  - testbench
  - interface
  - RTL
- Assertions are useful for:
  - protocol checking
  - design invariants
  - pipeline timing
  - illegal conditions
  - regression testing

Immediate vs Concurrent Assertions

- Immediate assertion (This is what we have used to far!)
  - checked at the point it executes
  - behaves like a procedural check
- Concurrent assertion
  - checked over time
  - synchronized to a clock
  - used for temporal behavior
- Examples:
  - Immediate: “sum must equal a+b right now”
  - Concurrent: “if req happens, ack must happen within 2 cycles”

Immediate assertion

- Good for combinational invariants
- Not ideal for temporal behavior
- Executes whenever the block executes

```
always_comb begin
    assert (sum == a + b)
        else $error("Adder mismatch");
end
```

Works for Combinational Logic!

How to verify the correctness of a DFF?

```
logic prev_d;

always_ff @(posedge clk) begin
    prev_d <= d;

    if (!rst) begin
        assert (q == prev_d)
    else $error("Flip-flop output mismatch");
    end
end
```

Store previous value

Challenging for Sequential Logic!

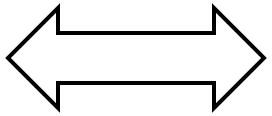
Basic Concurrent Assertion Syntax

- property ... endproperty
- assert property (...)
- Usually clocked using @(posedge clk)
- Disable during reset using disable iff (rst)

```
property p_name;
    @(posedge clk) disable iff (rst)
        antecedent |-> consequent;
endproperty

assert property (p_name);
```

Equivalent



```
assert property (@(posedge clk) disable iff (rst)
        antecedent |-> consequent);
```

Implication Operators

- |-> = overlapping implication
- |=> = non-overlapping implication
- Overlapping: consequent starts same cycle
- Non-overlapping: consequent starts next cycle

a	b	a -> b
0	0	T
0	1	T
1	0	F
1	1	T

Boolean Truth Table for Implication

|-> is NOT the same as ==
Different truth table!

- Examples
  - a |-> b means: if a is true now, b must be true now
  - a |=> b means: if a is true now, b must be true next cycle

Concurrent Assertion Example

- Example 1:
  - Combinational feed through (wire)

```
assert property (@(posedge clk) disable iff (rst)
        out == in
);
```

- Example 2:
  - If `en` is true in this cycle, then `valid` must also be true in this cycle.

```
assert property (@(posedge clk) en |-> valid);
```

- Example 3:
  - If **`req`** is high on a clock edge, **`ack`** must be high on the next edge
  - This is a one-cycle response property

```
assert property (@(posedge clk) disable iff (rst)
        req |=> ack
);
```

Additional Temporal Building Blocks

- $past
  - $past(sig) sampled value from previous cycle.
  - $past(sig,2) sampled value from 2 cycles ago.
- $stable
  - $stable(sig) no change since previous sample
- $rose
  - $rose(sig) signal changed 0 to 1
- $fell
  - $fell(sig) signal changed 1 to 0

```
assert property (@(posedge clk)
    q == $past(d,1));
```

```
assert property (@(posedge clk)  $stable(q);
```

```
assert property (@(posedge clk) disable iff (rst)
     $rose(start) |=> busy
);
```

When Are Assertions Evaluated? (Clock Semantics)

- Concurrent assertions are evaluated at a clock event
  - Typically: `@(posedge clk)`
- Step 1: Signals are sampled at the clock edge
  - All signals used in the assertion are captured (sampled) at this instant
- Step 2: Assertion evaluates using sampled values
  - Uses a consistent snapshot of the design at that edge
- Step 3: RTL registers update *after* sampling
  - Non-blocking assignments (<=) update in the NBA region
  - This happens after assertion sampling

Debugging Assertions (Clock & Print Behavior)

- Assertions are evaluated at clock edges
  - Typically: `@(posedge clk)`
- Implication of `d |=> q`
  - `d` is sampled at **cycle N**
  - `q` is checked at **cycle N+1**
- Print statements follow the evaluation time
  - `$display` / `$error` triggers when the assertion is evaluated
  - For `|=>`, this happens on the **next clock edge (N+1)**
- Common confusion
  - You see a failure at cycle N+1
  - But the cause (d) occurred at cycle N

```
assert property (@(posedge clk)
    d |=> q)
else begin
    $error("ASSERT FAIL @%0t: q=%0b,
    d previous cycle=%0b)",
    $time, q, $past(d,1));
end
```

In-Class Demo

- 4-2 Encoder – Immediate Assertions
  - This works fine
- 4-2 Encoder – Concurrent Assertions
  - Similar complexity but decouples the code
- Flip-Flop - Concurrent Assertions
  - Significant advantage compared with immediate assertions
  - Practice using implication =>, disable iff, $past, and $stable

Pitfalls with Assertion and Reset

- Which of these will work for a DFF?
- The challenge is the first cycle after reset

1. assert property (@(posedge clk) disable iff (rst) in |=> out);
2. assert property (@(posedge clk) disable iff (rst) out == $past(in, 1));
3. assert property (@(posedge clk) disable iff (rst) 1 |=> out == $past(in, 1));
4. assert property (@(posedge clk) !rst |=> out == $past(in, 1));

Signal	Clock cycle			
	t1	t2	t3	t4
rst	1	1	0	0
in	1	1	1	1
out	x	0	0	1

One answer is incomplete, one is wrong, and two are correct and equivalent